Japanese engineers) with Japanese language support in mind (hence the esoteric `mb_convert_kana()`), and it functions best when left as language-agnostic. (No disrespect to the authors of `mbstring`; it's still fantastically useful!) The settings you're most likely to ever need to care about are detailed below.

`internal_encoding`

`mbstring` processes text using an internal character encoding. Any incoming data that is in a different format will be transformed to the internal encoding before processing (any return data will be transformed back to match the input). Note that using `mbstring.internal_encoding` in an `.ini` file is deprecated; you should use PHP's core `default_charset` setting instead. Can be set at runtime with `mb_internal_encoding()`.

`detect_order`

Many `mbstring` functions contain internal logic for automatically detecting encoding. Various encoding systems will be tried; this setting defines in what order those attempts are made. Values can be either `auto` (default order) or any available encoding name. Multiple values are comma separated. Runtime configuration is available via `mb_detect_order()`.

`substitute_character`

If an invalid byte sequence is encountered by `mbstring` functions, or a decoded codepoint does not exist in the output encoding, it is swapped with the substitution character specified here. Values can be an integer ordinal Unicode codepoint or one of `"none"`, `"long"`, or `"entity"`. The runtime function is `mb_substitute_character()`; details follow further on in this section.

`strict_detection`

Controls the encoding detection behavior of `mb_detect_encoding()`. Detecting encoding is usually an operation of last resort; it is far better to specify these things in configuration, HTTP headers, and so forth. A non-strict detection will match the closest from a list; strict requires that there's an exact match. We'll see this in action shortly when we look at `mb_detect_encoding()`.

Hopefully some of the above already makes sense, having seen many `mbstring` functions at work in the previous section. Now, on with the show… Let's dive into more functions!